APPRISE WORDPRESS INTEGRATION PROJECT

Network Programming Project Report

Student	Fadi Omar
Project Type	WordPress - Apprise - Telegram Integration
Platform	Proxmox VE / Ubuntu Server / Docker
Repository	https://github.com/fadiomar1/project-network-programming
Public URL	https://fadi-apprise.shares.zrok.io

Submitted as a complete project package containing the report, presentation slides, source code, YAML files, and demonstration material.

Abstract

This project implements a complete notification integration platform that connects WordPress with the Apprise API and Telegram. The system is hosted on an Ubuntu Server virtual machine inside Proxmox VE and deployed using Docker containers. A custom WordPress plugin collects notification information and sends it to Apprise through an HTTP POST request. Apprise then forwards the notification to a Telegram bot. Containerlab is used to build and test a routed network topology, while zrok publishes the local WordPress service through a fixed public HTTPS URL. The project demonstrates virtualization, containerization, API integration, network topology automation, reverse proxy handling, and public service exposure.

Table of Contents

1. Introduction

Modern web systems frequently need to send notifications to external services. WordPress provides a flexible plugin architecture, while Apprise provides a unified notification gateway that supports many services such as Telegram, Discord, Slack, email, and others. This project combines both systems in a practical network programming environment.

2. Project Objectives

The main objective is to build and demonstrate a working integration that can be accessed locally and publicly, while keeping each component isolated and manageable.

- Deploy WordPress and MariaDB using Docker Compose.
- Deploy the Apprise API as an independent Docker service.
- Create a custom WordPress plugin that sends notifications using HTTP POST requests.
- Deliver notifications to Telegram through an Apprise configuration.
- Create a multi-node Containerlab topology using YAML.
- Publish WordPress through a fixed zrok public URL.
- Verify the system using HTTP status checks, container status, ping tests, and real Telegram messages.

3. System Architecture

The system is organized into several layers. Proxmox provides virtualization, Ubuntu Server hosts the project, Docker runs the application services, WordPress provides the user interface, the custom plugin performs the API request, Apprise handles notification delivery, and Telegram receives the final message.

Logical data flow

```
Internet User
    |
    v
zrok HTTPS Public URL
    |
    v
WordPress + Custom Plugin (Docker, port 8080)
    |  HTTP POST /notify/{configuration_id}
    v
Apprise API (Docker, port 8000)
    |
    v
Telegram Bot -> Telegram Notification
```

- The WordPress front end is available to general visitors.
- The WordPress administration page is protected by authentication.
- The plugin communicates with Apprise through the Docker host gateway.
- The public URL is mapped to the local WordPress service using zrok.

4. Infrastructure and Deployment

4.1 Proxmox VE and Ubuntu Server

Proxmox VE is used as the virtualization platform. An Ubuntu Server VM is configured to start automatically and is connected to an internal bridge network. The Ubuntu VM uses the internal address 10.10.10.10, while Proxmox forwards selected external ports to the VM.

4.2 Docker Services

- wordpress - Apache and PHP application container.
- wordpress-db - MariaDB database container.
- apprise - Apprise API container listening on port 8000.
- Containerlab nodes - PC1, R1, and PC2 containers.

Service verification commands

```
docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"

docker inspect -f '{{.Name}} -> {{.HostConfig.RestartPolicy.Name}}' apprise wordpress wordpress-db
```

4.3 WordPress Docker Compose

docker-compose.yml (credentials represented as environment variables)

```
services:
  database:
    image: mariadb:11
    container_name: wordpress-db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: ${WORDPRESS_DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
    container_name: wordpress
    restart: unless-stopped
    ports:
      - "8080:80"
    depends_on:
      - database
    volumes:
      - ./wordpress:/var/www/html
```

4.4 Apprise Docker Compose

apprise-docker-compose.yml

```
services:
  apprise:
    image: caronc/apprise:latest
```

```
  container_name: apprise
  restart: unless-stopped
  ports:
   - "8000:8000"
  environment:
    APPRISE_STATEFUL_MODE: simple
    APPRISE_WORKER_COUNT: "1"
    APPRISE_ADMIN: "y"
    TZ: Asia/Hebron
  volumes:
   - ./config:/config
   - ./plugin:/plugin
   - ./attach:/attach
```

5. Containerlab Network Topology

Containerlab is used to define a reproducible routed network. PC1 and PC2 are placed in two separate IPv4 networks and communicate through router R1. IPv4 forwarding is enabled on the router.

Topology diagram

```
PC1 192.168.10.2/24 ---- R1 ---- 192.168.20.2/24 PC2
                         |
            eth1: 192.168.10.1/24
            eth2: 192.168.20.1/24
```

router.clab.yml

```
name: router-lab

topology:
  nodes:
    pc1:
      kind: linux
      image: alpine:latest
      exec:
        - ip link set eth1 up
        - ip addr add 192.168.10.2/24 dev eth1
        - ip route replace 192.168.20.0/24 via 192.168.10.1

    r1:
      kind: linux
      image: alpine:latest
      sysctls:
        net.ipv4.ip_forward: 1
      exec:
        - ip link set eth1 up
        - ip link set eth2 up
        - ip addr add 192.168.10.1/24 dev eth1
        - ip addr add 192.168.20.1/24 dev eth2

    pc2:
      kind: linux
      image: alpine:latest
      exec:
        - ip link set eth1 up
```

```
      - ip addr add 192.168.20.2/24 dev eth1
      - ip route replace 192.168.10.0/24 via 192.168.20.1

  links:
    - endpoints: ["pc1:eth1", "r1:eth1"]
    - endpoints: ["r1:eth2", "pc2:eth1"]
```

Topology deployment and connectivity test

```
containerlab deploy -t router.clab.yml
containerlab inspect -t router.clab.yml
docker exec clab-router-lab-pc1 ping -c 4 192.168.20.2
```

6. WordPress and Apprise Integration

6.1 Plugin Interface

The custom plugin adds an Apprise Integration settings page to WordPress. The administrator saves the Apprise API URL and configuration ID, then enters a notification title, message, and type.

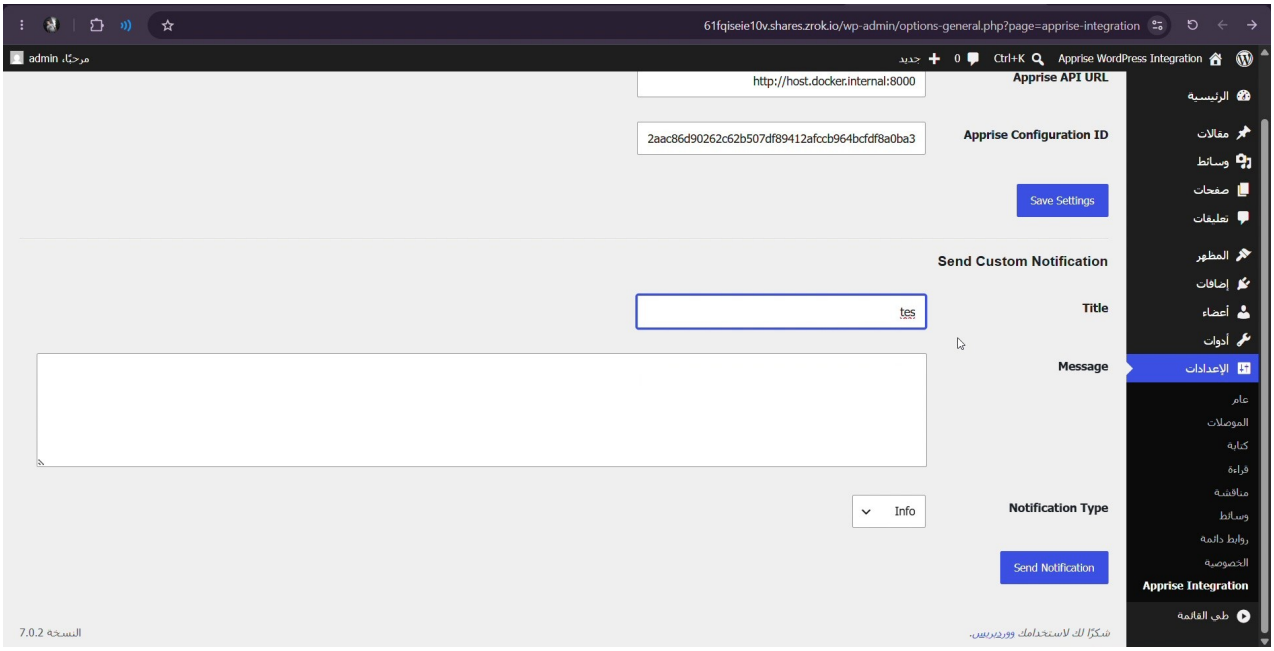

Figure 1. Custom WordPress Apprise Integration page.

6.2 Plugin Communication Logic

Core plugin HTTP POST logic

```
function fadi_apprise_send($title, $message, $type = 'info') {
    $api_url = rtrim(get_option(
        'fadi_apprise_api_url',
        'http://host.docker.internal:8000'
    ), '/');

    $config_key = get_option('fadi_apprise_config_key', '');
    $endpoint = $api_url . '/notify/' . rawurlencode($config_key);
```

```
    return wp_remote_post($endpoint, array(
        'timeout' => 20,
        'headers' => array('Content-Type' => 'application/json'),
        'body' => wp_json_encode(array(
            'title' => $title,
            'body'  => $message,
            'type'  => $type
        ))
    ));
}
```

The plugin uses the WordPress HTTP API rather than a raw socket. This allows WordPress to handle request creation, timeout management, and response errors. The JSON body contains the title, body, and notification type.

6.3 Telegram Configuration

Apprise stores a Telegram notification URL in a configuration identified by a configuration ID. The Telegram token and chat ID are not included in the public repository. When WordPress sends a request to the configured Apprise endpoint, Apprise uses the saved Telegram destination and sends the message.

Figure 2. Telegram notification received from the WordPress integration.

7. Public Access Using zrok

The WordPress service runs locally on port 8080. zrok is used to expose this local service through a public HTTPS address. A reserved name was created so the URL remains consistent between demonstrations.

Fixed public URL commands

```
zrok create name fadi-apprise
zrok list names
zrok share public http://127.0.0.1:8080 -n public:fadi-apprise
```

- Reserved public name: fadi-apprise
- Public website: https://fadi-apprise.shares.zrok.io
- Public administration page: https://fadi-apprise.shares.zrok.io/wp-admin
- The zrok share process must be running unless configured as an automatic service.

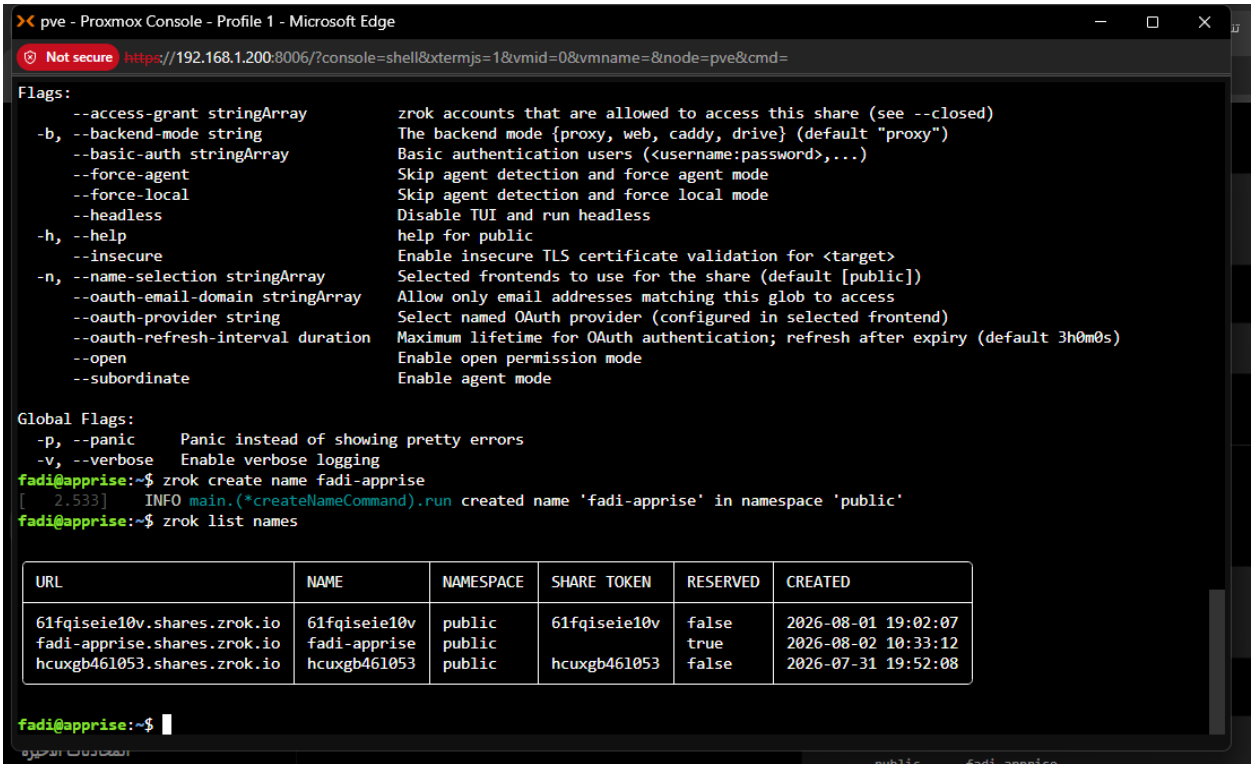

Figure 3. Reserved zrok name displayed by zrok list names.

7.1 Reverse Proxy Handling

wp-config.php reverse proxy adjustment

```
if (!empty($_SERVER["HTTP_HOST"]) &&
    strpos($_SERVER["HTTP_HOST"], ".shares.zrok.io") !== false) {
    $zrok_host = preg_replace("/:\d+$/", "", $_SERVER["HTTP_HOST"]);
    $_SERVER["HTTP_HOST"] = $zrok_host;
    $_SERVER["SERVER_NAME"] = $zrok_host;
    $_SERVER["SERVER_PORT"] = 443;
    $_SERVER["HTTPS"] = "on";
    define("WP_HOME", "https://" . $zrok_host);
    define("WP_SITEURL", "https://" . $zrok_host);
}
```

This adjustment prevents WordPress from redirecting public users to the internal port 8080 and ensures that generated URLs use HTTPS.

8. Testing and Verification

The project was tested at every layer. The following checks provide evidence that each component works correctly.

Test	Command / Action	Expected Result
Docker containers	docker ps	Containers show Up / running
WordPress local	curl -I http://127.0.0.1:8080	HTTP/1.1 200 OK
Apprise status	curl http://127.0.0.1:8000/status	OK
Containerlab routing	ping from PC1 to 192.168.20.2	0% packet loss
zrok public URL	curl -I https://fadi-apprise.shares.zrok.io	HTTP/2 200
Telegram delivery	Send custom notification in WordPress	Message arrives in Telegram

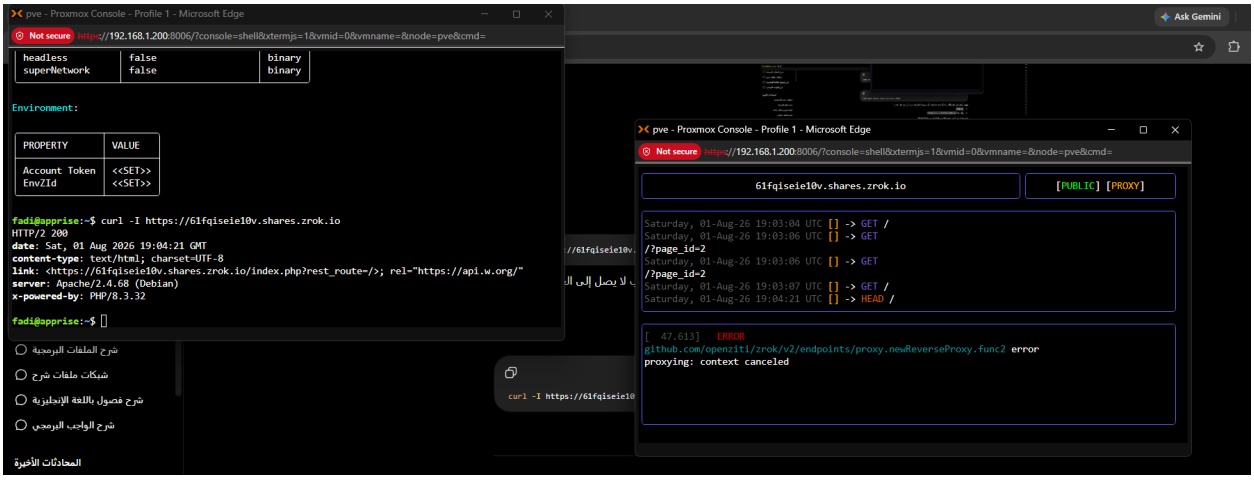

Figure 4. zrok requests and successful HTTP/2 200 verification.

9. Security and Reliability

- Secrets such as database passwords, Telegram tokens, and zrok account tokens must not be committed to GitHub.
- The .gitignore file excludes WordPress runtime files and sensitive configuration.
- The WordPress administration page requires authentication.
- Docker restart policies use unless-stopped to restore services after reboot.
- The Ubuntu VM is configured for automatic startup in Proxmox.
- The zrok public name is fixed, but the share should be stopped when public access is not required.

Project credentials should be provided privately to the instructor. They are intentionally excluded from this public report and repository.

10. Results

- WordPress and MariaDB run successfully in Docker.
- The custom WordPress plugin sends structured JSON notifications.
- Apprise receives the request and forwards it to Telegram.
- The Telegram bot receives the message in real time.
- The Containerlab topology routes traffic between two separate networks.
- The WordPress site is accessible using a fixed public zrok URL.
- All main source files are stored in the GitHub repository.

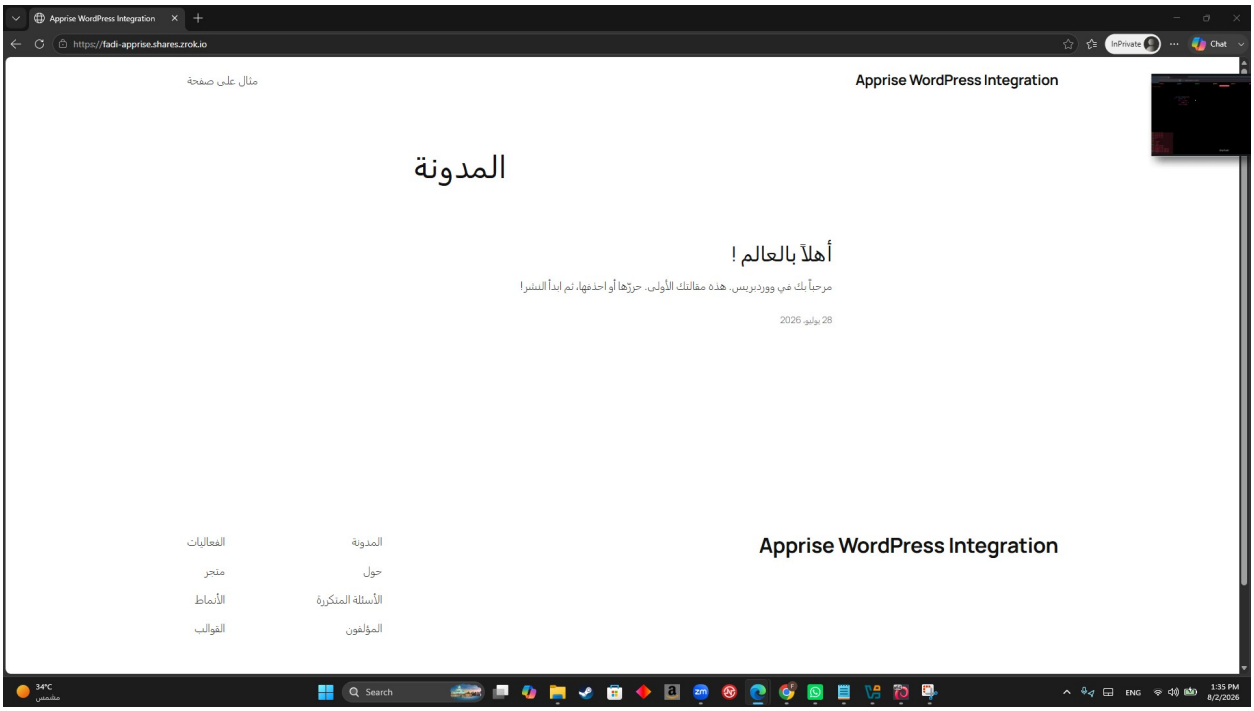

Figure 5. WordPress site available through the fixed public zrok URL.

11. Demonstration Video

An accompanying demonstration video explains and verifies the complete implementation. The video shows the project environment, Docker containers, YAML files, WordPress plugin, Apprise interface, Telegram notification, and zrok public link. The report, presentation slides, code, designs, and supporting materials are prepared for submission with the project package.

- Proxmox VE and Ubuntu Server startup.
- Docker and Containerlab status.
- Review of docker-compose.yml and router.clab.yml.
- WordPress login and Apprise Integration page.
- Sending a custom notification.
- Telegram message arrival.
- Public access through the fixed zrok URL.

12. Conclusion and Future Work

The project successfully combines virtualization, containerization, routing, web development, and external notification services. It provides a complete example of how a private application can communicate with an API and be made publicly available in a controlled way. The design is modular because WordPress, Apprise, MariaDB, Containerlab, and zrok can be managed independently.

Future Improvements

- Run zrok as a systemd service for automatic public availability.
- Add role-based access control to the WordPress plugin.
- Support additional Apprise destinations such as Discord, Slack, and email.
- Trigger automatic notifications when posts are published or forms are submitted.
- Move sensitive values to environment variables or Docker secrets.
- Add automated tests and GitHub Actions validation.

Appendix A - Main Commands

```
# Connect to Ubuntu
ssh fadi@10.10.10.10

# Check containers
docker ps

# WordPress status
curl -I http://127.0.0.1:8080

# Apprise status
curl http://127.0.0.1:8000/status

# Containerlab
containerlab inspect -t ~/router-lab/router.clab.yml

# Start fixed zrok share
zrok share public http://127.0.0.1:8080 -n public:fadi-apprise

# Public test
curl -I https://fadi-apprise.shares.zrok.io
```

Appendix B - Source Files and URLs

- GitHub repository: https://github.com/fadiomar1/project-network-programming
- Public WordPress: https://fadi-apprise.shares.zrok.io
- Public WordPress Admin: https://fadi-apprise.shares.zrok.io/wp-admin
- Local WordPress Admin: http://192.168.1.200:8080/wp-admin
- Local Apprise Interface: http://192.168.1.200:8000
- Local Proxmox Interface: https://192.168.1.200:8006

Note: Local URLs are accessible only when the project host is running and the client is connected to the same local network. Public access requires the zrok share process to be active.